Coding Conventions: Naming, Layout and Register Rules

A companion guide to Building an AMIGA Game Engine

Richard Shannon

AmigaGameEngine v1.0

Contents

1. Files 1

2. Names 2

3. Layout 2

4. Routine Headers 2

5. Registers 3
Mind the condition codes 3

6. Engine and Game Boundaries 3

7. Hardware Access 3

8. Branches and Warnings 4

9. Python 4

10. Before You Commit 4

The engine is one program assembled in a single pass (see the Single Translation Units Guide). Every global label is visible everywhere, and no linker or type system separates the engine from the game. What keeps 35,000 lines of assembly readable and safe to change is a small set of rules that every file follows. This guide lists them, with the reason for each.

The rules apply to everything under `engine/` and `games/`. Third-party code keeps its upstream style and is exempt: `engine/audio/ptplayer/` (Frank Wille's PTPlayer) and `engine/system/zx0_faster.asm` (the ZX0 decompressor).

1. Files

Kind	Style	Example
Assembly source	`snake_case.asm`	`bob_compositor.asm`, `player_fsm.asm`
Assembly header (equates and structs, no code)	`snake_case.i`	`engine_const.i`, `actor_struct.i`
Python tool	`snake_case.py`	`export_level.py`
Generated data	as the generator emits it, with an "Auto-generated" banner	`level_01_entities.asm`

Every file opens with the engine banner: `AMIGA GAME ENGINE`, the file name, and a one-line purpose.

Keep each file's existing line endings. The repository mixes CRLF and LF files, and converting a whole file buries the real change in the diff.

2. Names

Kind	Style	Examples
Routine (global label)	`CamelCase`, prefixed by the full subsystem or screen name	`TilemapDrawPlayer`, `LevelCompleteGenPassword`, `VhsStartEffect`, `CharBltString`
Local label	`.snake_case`	`.erase_loop`, `.done_erase_enemies`
Variable (`Variables` block field)	`CamelCase`, no underscore	`WaterPixelY`, `VhsFrameCount`, `SavedINTENA`
Struct field	`Struct_Field`	`Player_X`, `Actor_PrevY`, `Enemy_Speed`, `Snap_PlayerX`
Struct size and padding	`Struct_Sizeof`, `Struct_Pad`	`Player_Sizeof`, `Variables_Sizeof`
Constant	`UPPER_SNAKE`, prefixed by subsystem	`VHS_EFFECT_DURATION`, `PLAYER_WIDTH`
Bit number / bit mask	`FOOB_X` / `FOOF_X`	`INTB_VERTB`, `INTF_VERTB`
Macro	`UPPER_SNAKE`	`PUSHM`, `WAITBLIT`, `PLAYSFX`, `KEYTEST`
Engine-to-game hook	`Game` + `CamelCase` + `Hook`	`GameLogicHook`, `GameSnapshotSaveHook`
Copper list	`cp` + `CamelCase`	`cpGame`, `cpTitlePal`, `cpSprites`
Data blob or variant	`Name_Variant`	`SfxCane_Raw`, `SfxCane_RawEnd`, `WaterWaveTable_Even`
Generated level data	`<Prefix>_<Field>`, prefix from the `.tmx` name	`Level_01_PlayerStartX`

Write acronyms as words inside CamelCase (`Vhs`, `Blt`, `Dos`). Hardware register and AmigaOS names keep their usual spelling (`SavedINTENA`, `SavedDOSBase`, `cpGameDIW`).

Why full prefixes: with one global namespace, the prefix is the only thing that says where a routine lives, and it is what you grep for. A routine private to one screen still carries that screen's full name (`TitleDrawMenu`, not `TS_DrawMenu`).

3. Layout

- Labels start in column 0, mnemonics in column 4, operands in column 16. Trailing comments line up near column 40:

```
TilemapCentredCameraY:
    move.w      Player_Y(a4),d1
    lsl.w       #4,d1
    add.w       Player_YDec(a4),d1      ; d1 = PlayerPixelY
```

- Directives are lowercase: `include`, `incdir`, `section`, `dc.w`, `ds.b`, `even`, `rsreset`, `rs.w`. Conditional-assembly and macro keywords are uppercase: `IFND`, `IFEQ`, `ENDC`, `MACRO`, `ENDM`.
- Define constants with `=`, never `equ`.
- Use the register names from `amiga_hw.i` with `CUSTOM` or `(a6)`, never a raw `$dffXXX` address.

4. Routine Headers

Every global routine starts with a header block: what it does, what it expects in registers, what it returns, and what it destroys.

```
;===============================================================================
; TilemapCentredCameraY  -  Camera Y that centres the player in the view
;
; In:  a4 = player struct
; Out: d1.w = PlayerPixelY - CAM_CENTRE_OFFSET, clamped to
;             [LevelMinCameraY .. LevelMaxCameraY]
; Destroys: none other than d1
;===============================================================================
```

`Destroys:` is a promise. A caller decides what to save from it, so keep it accurate whenever the routine changes. If a routine saves everything it uses, say `Destroys: none (preserves all registers)`. Explain *why* in comments, not just what. The hardware reasons (beam position, DMA slots, register side effects) are exactly what the next reader cannot see.

5. Registers

These hold for the whole program after `Restart`:

Register	Holds	Rule
`a5`	the `Variables` block	never used as scratch
`a6`	`CUSTOM` (`$dff000`)	never used as scratch
`a4`	the current player struct	set per call with `lea Player(a5),a4`
`a3`	the current actor struct	set per call
`sp`	`GameStack` (4 KB)	balanced in every routine

Save registers with the macros from `engine/core/macros.asm`: `PUSHM/POPM` (a `movem.l` list), `PUSHMOST/POPMOST` (`d0-a4`) and `PUSHALL/POPALL`.

Mind the condition codes

`movem` does not change the flags, but `move` does. So `move.l (sp)+,d0` to restore a register **after** a compare or test destroys the result you were about to branch on. Branch first, then restore on each path:

```
    tst.w       d1
    beq.s       .zero
    move.l      (sp)+,d0                ; restore on this path
    ...
.zero:
    move.l      (sp)+,d0                ; and on this one
```

Do not rely on a one-register `movem` to keep the flags either: an assembler is free to turn it into a `move`.

6. Engine and Game Boundaries

- **Engine code never names a game symbol** except through the contract: the `Player` struct, the constants listed in `ENGINE_CONTRACT.md`, the `Game*Hook` routines, the `AudioPlay*` routines and the data bindings. Anything else a feature needs from the game goes through a new hook. `tests/test_engine_contract.py` enforces this.
- **No backward-compatibility aliases.** When a symbol is renamed, rename every use. The only allowed form of `Name = OtherSymbol` is a *contract binding* in the game (`GameLogicHook = PlayerLogic`, `LevelDefaultPalette = Level_01_Palette`), placed in `game_hooks.asm` or next to the data in `main.asm`.
- **Nothing included before `engine_main.asm` emits bytes;** the first byte of the code hunk is the program's entry point.
- **No routine falls through into the next.** The last instruction of every code file is `rts`, `bra`, `jmp` or data. A routine that relied on falling into its neighbour breaks silently when one of them moves to another file. The contract test checks this as well.

7. Hardware Access

- **Wait for the Blitter before touching its registers:** `WAITBLIT` before the first register write of every blit. The macro also does a dummy `DMACONR` read, which on early Agnus revisions gives the busy flag time to become valid.
- **Draw in the order of the Blit Order Guide.** A blit placed in the wrong phase tears even when its code is correct.
- **Address the ring buffer by the Ring Buffer Guide:** draw-time line for moving objects, both halves for background.

- **Size tables from the engine limits** (`MAX_GAME_MAP_SIZE`, `MAX_ACTIVE_*`), and clamp loops with the live value (`CurrentMapHeight`), never one level's size.
- **Copper lists reset `FMODE`, `BPLCON3` and `BPLCON4`** at the top, so AGA machines show the OCS layout.

8. Branches and Warnings

- Let `vasm` choose branch sizes where it can. Write `.s` only where you know the target is near, and never branch to the very next instruction (`vasm` warns, 2058); delete the branch instead.
- The build suppresses two warnings on purpose (`-nowarn=2047`, `-nowarn=2069`). Treat every other warning as an error to fix.

9. Python

- `snake_case` functions and variables, `UPPER_SNAKE` module constants.
- A module docstring at the top saying what the script converts, from where, to where, and how to run it. `argparse` or `tool_cli.run_tool()` for options.
- Converters are deterministic: seed any randomness, sort any sets, always write the output. The converter golden test pins every output (Asset Pipeline Guide §9).
- Load the palette from `game_palette.json`; never hard-code colours.

10. Before You Commit

```
python -m unittest discover tests          # contract, converter and executable goldens
python build.py --game duck_rescue         # and every other game you touched
```

If the executable golden changes, test the build in the emulator first, then accept it with `python tests/update_golden.py --executables`.